

# **FATEC JACAREÍ**

Faculdade de Tecnologia de Jacareí

Error Squad

Felipe Pacheco | Leonardo Irineu | Carlos Eduardo

João Vitor | Arthur Facchinetti | Caio Araujo

## **INSERÇÃO DE DADOS E CONSULTAS**

Dia 3 – Inserção de Dados e Consultas



## SUMÁRIO

|     |                                 |    |
|-----|---------------------------------|----|
| 1   | Objetivo do Dia .....           | 3  |
| 2   | Inserção de Dados .....         | 3  |
| 2.1 | Inserção em lojas .....         | 3  |
| 2.2 | Inserção em usuarios .....      | 4  |
| 2.3 | Inserção em clientes .....      | 5  |
| 2.4 | Inserção em leads .....         | 6  |
| 2.5 | Inserção em negociacoes .....   | 7  |
| 2.6 | Inserção em logs .....          | 8  |
| 3   | Consultas Obrigatórias .....    | 9  |
| 3.1 | Operadores \$and e \$or .....   | 9  |
| 3.2 | Operadores \$gt e \$lt .....    | 9  |
| 3.3 | Operador \$exists .....         | 10 |
| 3.4 | Projeção e Ordenação .....      | 10 |
| 3.5 | Paginação .....                 | 11 |
| 4   | Aggregations de Dashboard ..... | 11 |
| 4.1 | Leads por Origem .....          | 11 |
| 4.2 | Leads por Status .....          | 12 |
| 4.3 | Taxa de Conversão .....         | 12 |
| 4.4 | Leads por Atendente .....       | 13 |
| 4.5 | Leads por Importância .....     | 13 |
| 5   | Conclusão .....                 | 14 |



## 1 OBJETIVO DO DIA

Este documento apresenta os procedimentos executados no MongoDB Compass para atender os requisitos de entrega do Dia 3 do projeto de banco de dados não relacional da 1000 Valle Multimarcas. As atividades contemplam:

- inserção dos dados nas coleções obrigatórias do modelo;
- aplicação das regras de negócio definidas na modelagem;
- execução das consultas obrigatórias com filtros, projeção, ordenação e paginação;
- geração de indicadores gerenciais por meio de operações de aggregation.

A estrutura do documento segue o padrão estabelecido nos dias anteriores, com explicações detalhadas de cada comando e suas justificativas técnicas.

## 2 INSERÇÃO DE DADOS

Nesta etapa foram populadas as coleções obrigatórias do projeto, seguindo a modelagem definida no Dia 2. As referências entre entidades reutilizáveis foram mantidas por meio de *referencing*, enquanto o histórico de negociação foi inserido por *embedding*, conforme decisão arquitetural documentada. As coleções populadas são: **lojas**, **usuarios**, **clientes**, **leads**, **negociacoes** e **logs**.

### 2.1 Inserção em `lojas`

O comando a seguir insere três unidades da 1000 Valle Multimarcas na coleção **lojas**. Os identificadores fixos garantem que todas as referências de leads e negociações possam ser corretamente vinculadas a cada unidade física.

```
db.lojas.insertMany([
  {
    _id: ObjectId("650000000000000000000001"),
    nome: "1000 Valle Centro",
    endereco: "Av. Brasil, 1234",
    telefone: "11 4002-8922",
    createdAt: new Date("2026-05-18T09:00:00Z")
  },
  {
    _id: ObjectId("650000000000000000000002"),
    nome: "1000 Valle Norte",
    endereco: "Rua das Palmeiras, 432",
    telefone: "11 4002-8923",
    createdAt: new Date("2026-05-18T09:10:00Z")
  },
  {
```

```

    _id: ObjectId("650000000000000000000003"),
    nome: "1000 Valle Sul",
    endereco: "Av. das Nações, 987",
    telefone: "11 4002-8924",
    createdAt: new Date("2026-05-18T09:20:00Z")
  }
]);

```

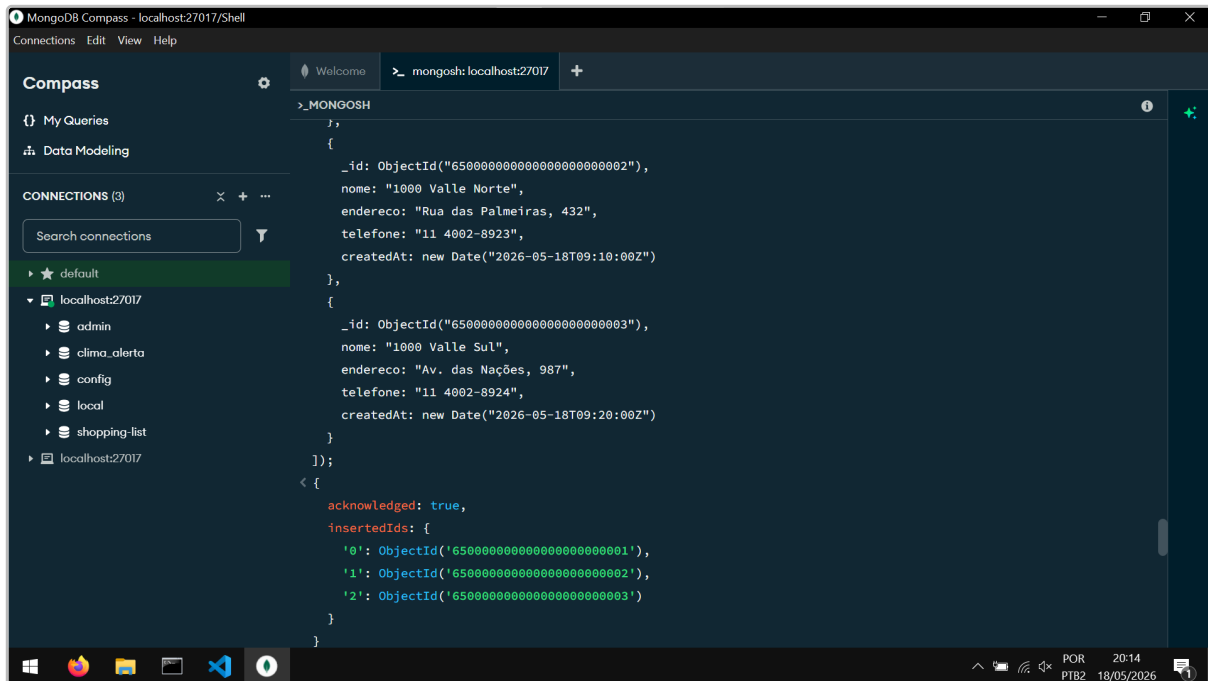


Figura 1 – Inserção de documentos na coleção lojas.

Fonte: elaborado pelos autores (2026).

## 2.2 Inserção em `usuários`

A coleção **usuários** reúne os atendentes, o gerente comercial e o administrador do sistema. O campo **cargo** identifica o papel de cada usuário no fluxo comercial e na trilha de auditoria dos logs.

```

db.usuarios.insertMany([
  {
    _id: ObjectId("650000000000000000000011"),
    nome: "Marcos Silva",
    cargo: "Atendente", area: "Vendas",
    email: "marcos.silva@1000valle.com",
    telefone: "11 99999-0001",
    createdAt: new Date("2026-05-18T09:30:00Z")
  },
  {
    _id: ObjectId("650000000000000000000012"),
    nome: "Ana Costa",
    cargo: "Gerente Comercial", area: "Vendas",
    email: "ana.costa@1000valle.com",
    telefone: "11 99999-0002",
    createdAt: new Date("2026-05-18T09:35:00Z")
  },
  {
    _id: ObjectId("650000000000000000000013"),
    nome: "Juliana Ramos",
    cargo: "Atendente", area: "Vendas",
    email: "juliana.ramos@1000valle.com",

```

```

    telefone: "11 99999-0003",
    createdAt: new Date("2026-05-18T09:40:00Z")
  },
  {
    _id: ObjectId("650000000000000000000014"),
    nome: "Pedro Almeida",
    cargo: "Atendente", area: "Vendas",
    email: "pedro.almeida@1000valle.com",
    telefone: "11 99999-0004",
    createdAt: new Date("2026-05-18T09:45:00Z")
  },
  {
    _id: ObjectId("650000000000000000000015"),
    nome: "Carla Souza",
    cargo: "Admin", area: "TI",
    email: "carla.souza@1000valle.com",
    telefone: "11 99999-0005",
    createdAt: new Date("2026-05-18T09:50:00Z")
  }
]);

```

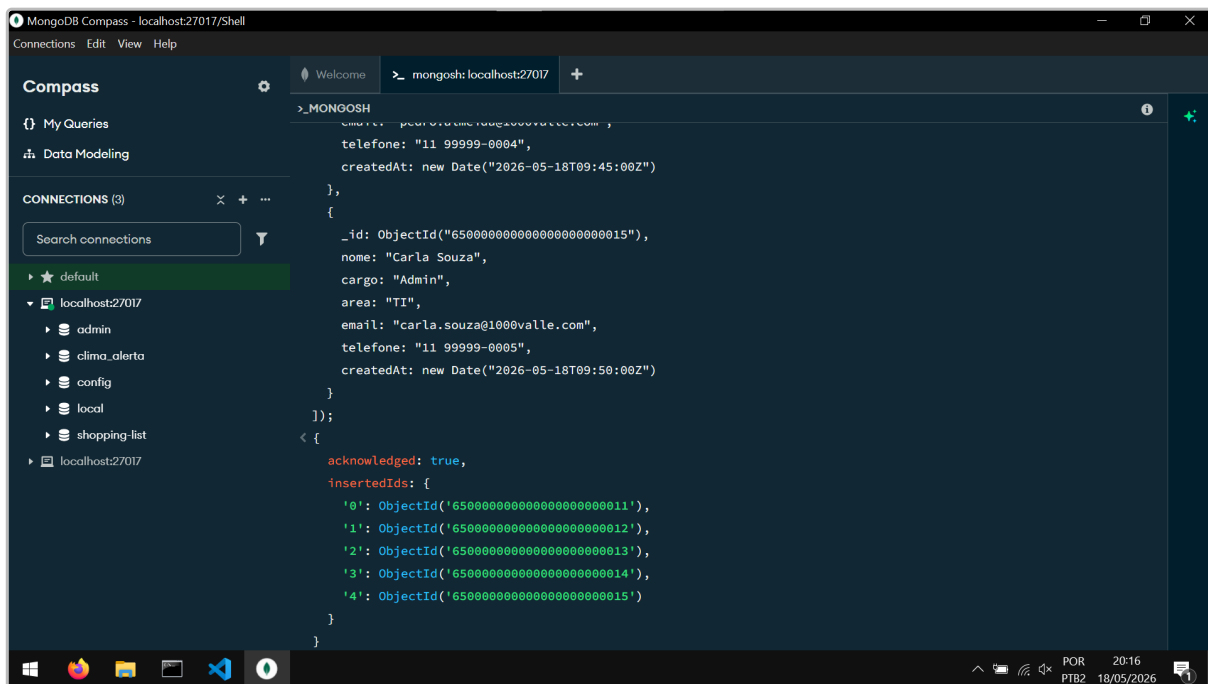


Figura 2 – Inserção de documentos na coleção usuarios.

Fonte: elaborado pelos autores (2026).

Os usuários cadastrados suportam o fluxo comercial completo e a auditoria das ações registradas na coleção logs.

## 2.3 Inserção em `clientes`

A coleção **clientes** armazena os dados cadastrais dos interessados. Cada cliente pode gerar múltiplos leads ao longo do tempo, razão pela qual a entidade é mantida de forma independente e vinculada aos leads pelo campo **clienteId**.

```
db.clientes.insertMany([
  {
    _id: ObjectId("650000000000000000000000000021"),
    nome: "Alice Fernandes",  cpf: "111.111.111-11",
    telefone: "11 98888-1111",
    email: "alice.fernandes@gmail.com",
    createdAt: new Date("2026-05-18T10:00:00Z")
  },
  {
    _id: ObjectId("650000000000000000000000000022"),
    nome: "Bruno Lima",  cpf: "222.222.222-22",
    telefone: "11 98888-2222",
    email: "bruno.lima@gmail.com",
    createdAt: new Date("2026-05-18T10:05:00Z")
  },
  {
    _id: ObjectId("650000000000000000000000000023"),
    nome: "Camila Sousa",  cpf: "333.333.333-33",
    telefone: "11 98888-3333",
    email: "camila.sousa@gmail.com",
    createdAt: new Date("2026-05-18T10:10:00Z")
  },
  {
    _id: ObjectId("650000000000000000000000000024"),
    nome: "Diego Pereira",  cpf: "444.444.444-44",
    telefone: "11 98888-4444",
    email: "diego.pereira@gmail.com",
    createdAt: new Date("2026-05-18T10:15:00Z")
  },
  {
    _id: ObjectId("650000000000000000000000000025"),
    nome: "Elisa Martins",  cpf: "555.555.555-55",
    telefone: "11 98888-5555",
    email: "elisa.martins@gmail.com",
    createdAt: new Date("2026-05-18T10:20:00Z")
  }
]);
```



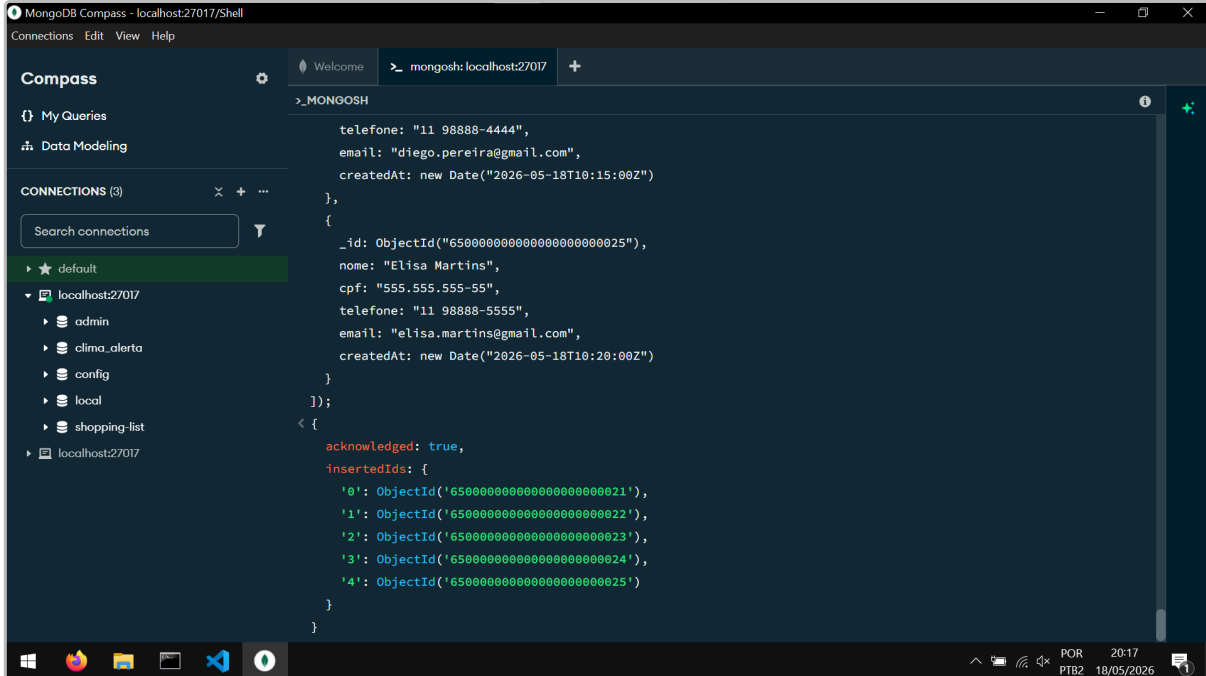


Figura 3 – Inserção de documentos na coleção clientes.

Fonte: elaborado pelos autores (2026).

Os dados de clientes permitem que cada lead esteja corretamente vinculado a uma pessoa real, evitando duplicidade de informações cadastrais.

## 2.4 Inserção em `leads`

A coleção **leads** representa as oportunidades comerciais do sistema. Cada documento referencia **clienteId**, **lojaId** e **atendenteId**, mantendo o vínculo com as entidades independentes por meio de *referencing*, conforme definido na modelagem do Dia 2.

```
db.leads.insertMany([
  {
    _id: ObjectId("65000000000000000000000031"),
    nomeLead: "Lead Alice - SUV",
    clienteId: ObjectId("65000000000000000000000021"),
    lojaId: ObjectId("65000000000000000000000001"),
    atendenteId: ObjectId("65000000000000000000000011"),
    origem: "whatsapp", status: "novo", importancia: "alta",
    createdAt: new Date("2026-05-18T10:30:00Z")
  },
  {
    _id: ObjectId("65000000000000000000000032"),
    nomeLead: "Lead Bruno - seminovo",
    clienteId: ObjectId("65000000000000000000000022"),
    lojaId: ObjectId("65000000000000000000000002"),
    atendenteId: ObjectId("65000000000000000000000013"),
    origem: "instagram", status: "em_atendimento", importancia: "media",
    createdAt: new Date("2026-05-18T10:35:00Z")
  },
  {
    _id: ObjectId("65000000000000000000000033"),
    nomeLead: "Lead Camila - hatch",
    clienteId: ObjectId("65000000000000000000000023"),
    lojaId: ObjectId("65000000000000000000000001"),
    atendenteId: ObjectId("65000000000000000000000014"),
```

```

origem: "telefone", status: "em_atendimento", importancia: "alta",
createdAt: new Date("2026-05-18T10:40:00Z")
},
{
  _id: ObjectId("650000000000000000000034"),
  nomeLead: "Lead Diego - troca",
  clienteId: ObjectId("650000000000000000000024"),
  lojaId: ObjectId("650000000000000000000003"),
  atendenteId: ObjectId("650000000000000000000011"),
  origem: "presencial", status: "convertido", importancia: "media",
  createdAt: new Date("2026-05-18T10:45:00Z")
},
{
  _id: ObjectId("650000000000000000000035"),
  nomeLead: "Lead Elisa - executivo",
  clienteId: ObjectId("650000000000000000000025"),
  lojaId: ObjectId("650000000000000000000002"),
  atendenteId: ObjectId("650000000000000000000013"),
  origem: "formulario", status: "perdido", importancia: "baixa",
  createdAt: new Date("2026-05-18T10:50:00Z")
}
]);

```

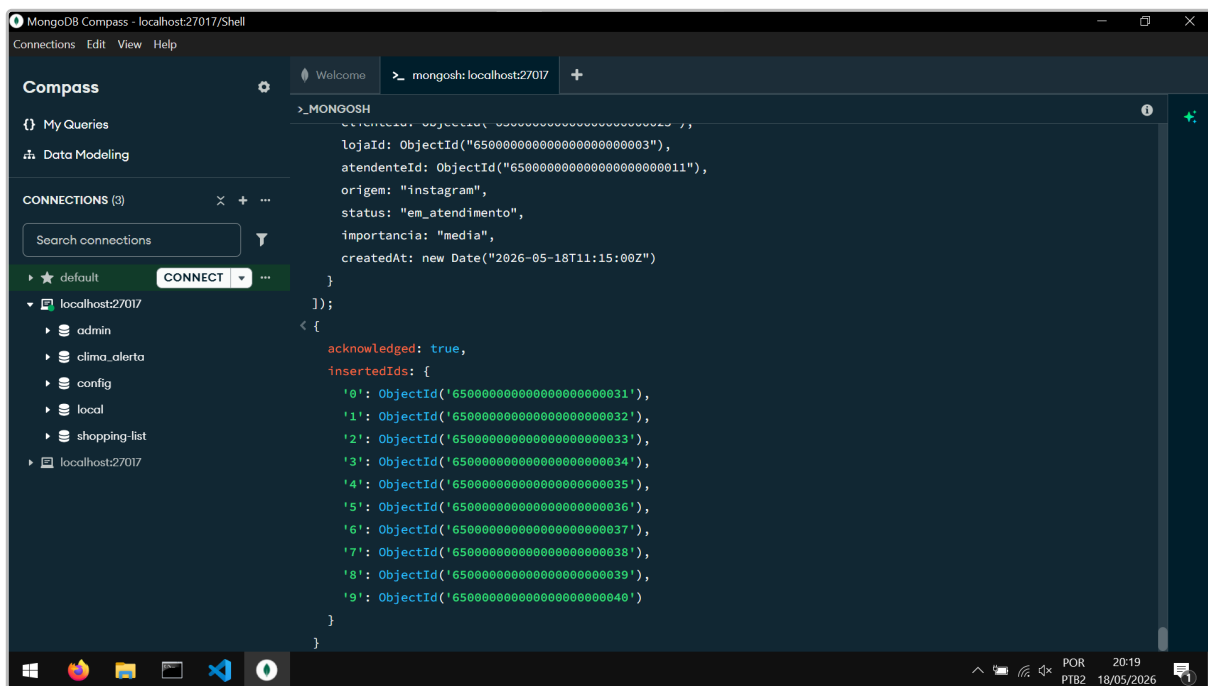


Figura 4 – Inserção de documentos na coleção leads.

Fonte: elaborado pelos autores (2026).

A modelagem garante:

- cada lead é vinculado a um cliente por clienteId;
- cada lead é vinculado a uma loja por lojaId;
- cada lead é vinculado a um atendente por atendenteId;
- o campo status controla o estágio do fluxo comercial.

## 2.5 Inserção em `negociacoes`

A coleção **negociacoes** registra o andamento de cada oportunidade comercial. A regra de apenas uma negociação ativa por lead é garantida pelo campo **ativo** e reforçada por índice único parcial. O histórico de eventos está embutido no documento por *embedding*, permitindo consultar a linha do tempo completa sem operações adicionais.

```
db.negociacoes.insertMany([
  {
    _id: ObjectId("65000000000000000000000051"),
    leadId: ObjectId("65000000000000000000000031"),
    status: "aberta",
    valorProposto: 45000, valorFinal: null, ativo: true,
    historico: [
      {
        data: new Date("2026-05-18T11:20:00Z"),
        etapa: "abertura",
        observacao: "Lead entrou pelo WhatsApp."
      }
    ],
    createdAt: new Date("2026-05-18T11:20:00Z")
  },
  {
    _id: ObjectId("65000000000000000000000052"),
    leadId: ObjectId("65000000000000000000000032"),
    status: "em_negociacao",
    valorProposto: 52000, valorFinal: null, ativo: true,
    historico: [
      {
        data: new Date("2026-05-18T11:25:00Z"),
        etapa: "primeira proposta",
        observacao: "Negociação em andamento via Instagram."
      }
    ],
    createdAt: new Date("2026-05-18T11:25:00Z")
  }
]);
```

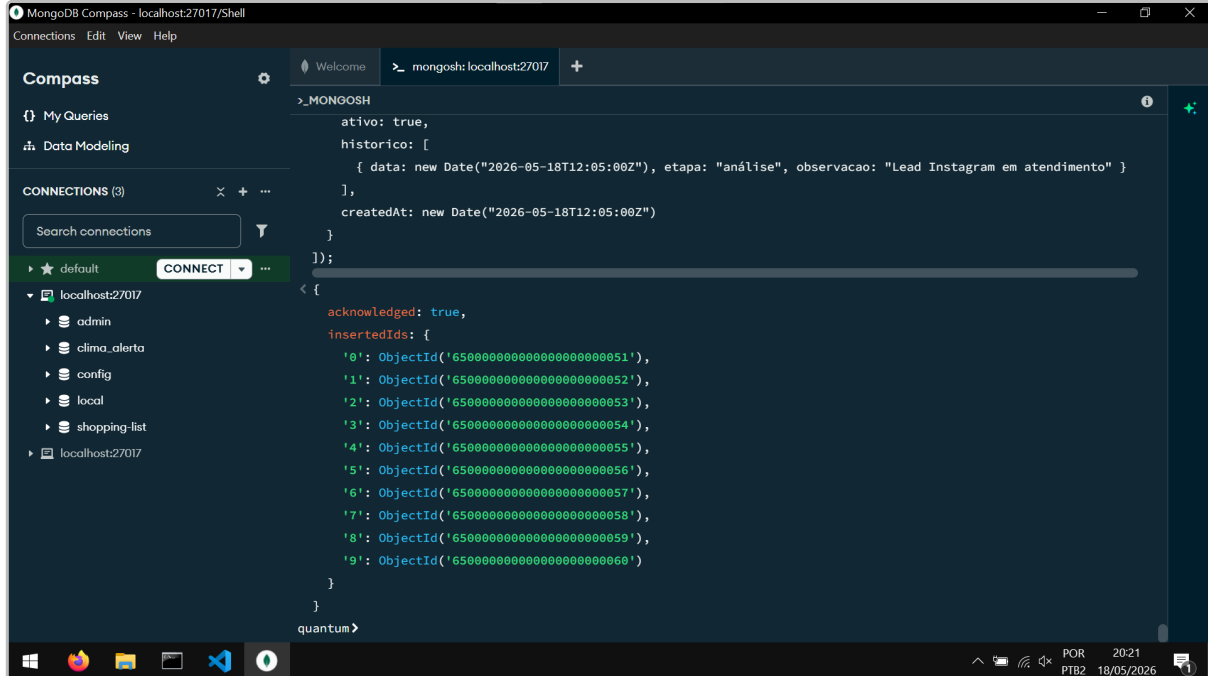


Figura 5 – Inserção de documentos na coleção negociacoes.

Fonte: elaborado pelos autores (2026).

O histórico embutido facilita as consultas operacionais sobre a evolução do processo de negociação, sem a necessidade de joins entre coleções.

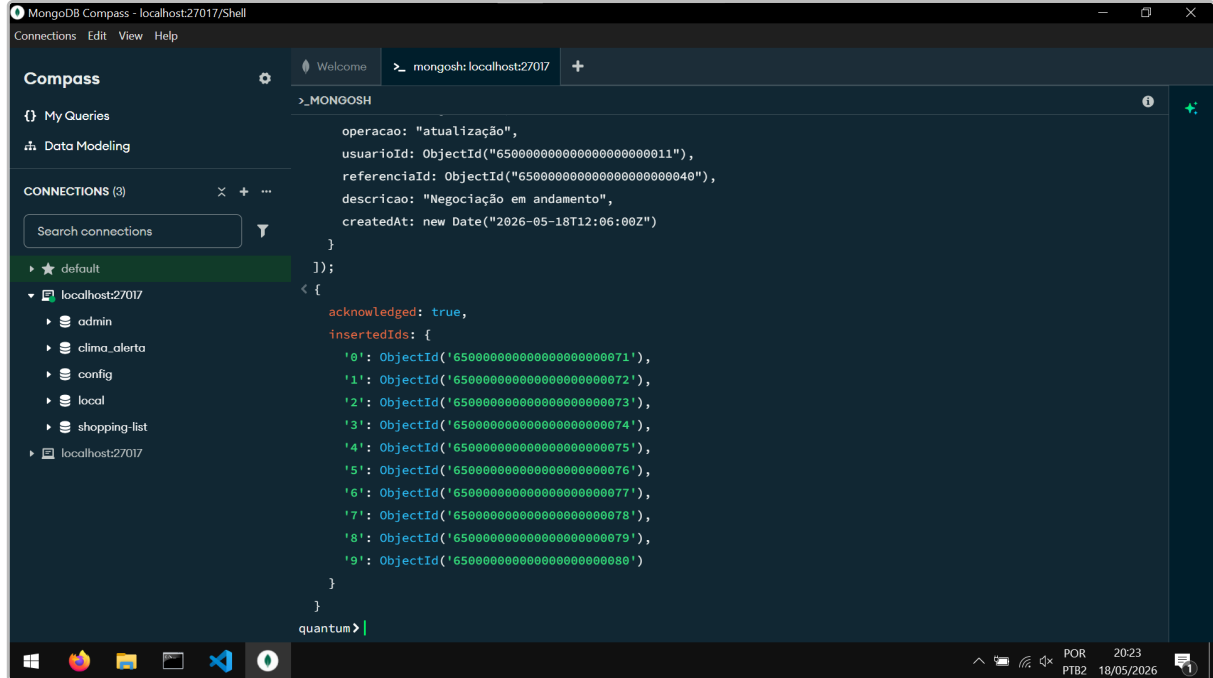
## 2.6 Inserção em `logs`

A coleção **logs** documenta as ações de criação e atualização de leads e negociações, fornecendo rastreabilidade completa das operações realizadas no sistema. Cada registro vincula a ação ao usuário responsável e à entidade afetada.

```

db.logs.insertMany([
  {
    _id: ObjectId("650000000000000000000071"),
    entidade: "lead", operacao: "criação",
    usuarioId: ObjectId("650000000000000000000011"),
    referenciaId: ObjectId("650000000000000000000031"),
    descricao: "Lead criado via WhatsApp",
    createdAt: new Date("2026-05-18T11:21:00Z")
  },
  {
    _id: ObjectId("650000000000000000000072"),
    entidade: "lead", operacao: "atualização",
    usuarioId: ObjectId("650000000000000000000013"),
    referenciaId: ObjectId("650000000000000000000032"),
    descricao: "Status alterado para em_atendimento",
    createdAt: new Date("2026-05-18T11:26:00Z")
  }
]);

```



*Figura 6 – Inserção de documentos na coleção logs.*

*Fonte: elaborado pelos autores (2026).*

Os registros de log permitem auditoria e rastreabilidade completa, confirmando cada operação realizada no sistema e identificando o usuário responsável.

### 3 CONSULTAS OBRIGATÓRIAS

Esta seção apresenta os comandos que demonstram os filtros e operações exigidos no requisito da atividade. Cada consulta é acompanhada da explicação técnica dos operadores utilizados.

#### 3.1 Operadores \$and e \$or

A consulta retorna leads que simultaneamente atendem às condições de status e de canal de origem. O operador **\$and** exige que todas as condições sejam verdadeiras, enquanto **\$or** aceita qualquer uma das origens especificadas.

```
db.leads.find({
  $and: [
    { status: { $in: ["novo", "em_atendimento"] } },
    { $or: [
      { origem: "whatsapp" },
      { origem: "instagram" }
    ]}
  ]
});
```

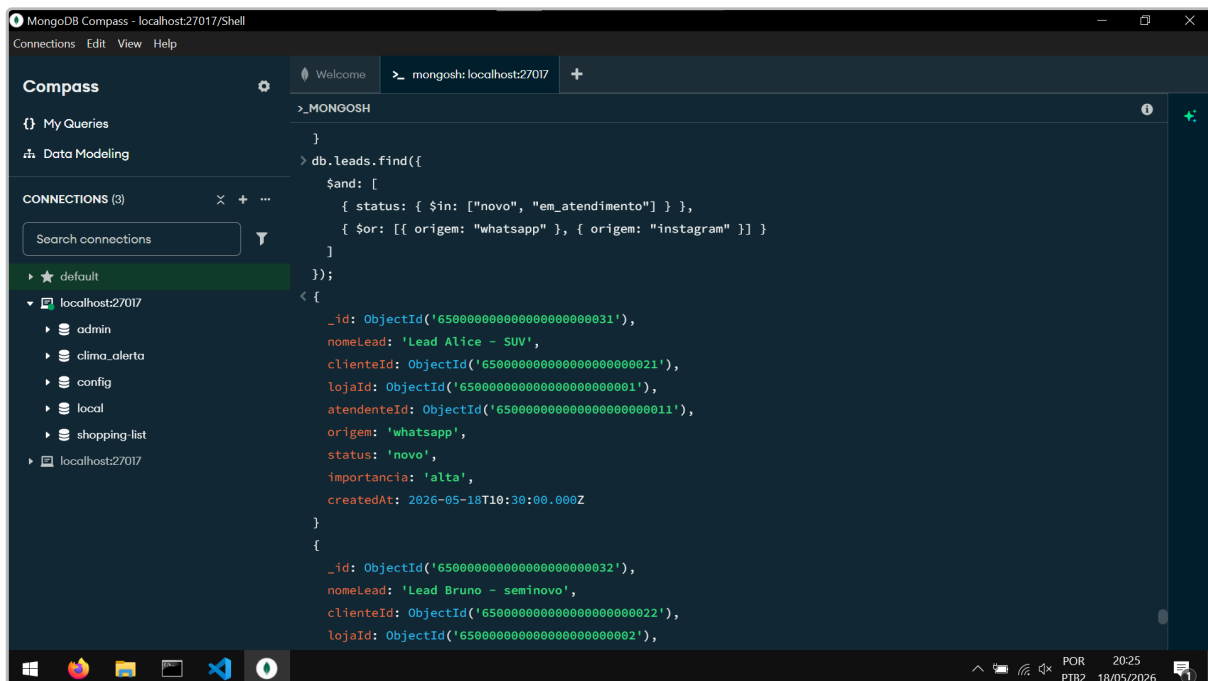


Figura 7 – Consulta com operadores \$and, \$or e \$in.

Fonte: elaborado pelos autores (2026).

- \$and: define condições simultâneas que devem ser todas satisfeitas;
- \$or: aceita documentos que atendam a qualquer uma das origens listadas;

- `$in`: filtra os valores de status de forma compacta.

### 3.2 Operadores `$gt` e `$lt`

A consulta localiza negociações cujo valor proposto encontra-se dentro de um intervalo numérico definido. Os operadores de comparação permitem filtrar documentos por faixas de valor.

```
db.negociacoes.find({
  valorProposto: { $gt: 30000, $lt: 120000 }
});
```

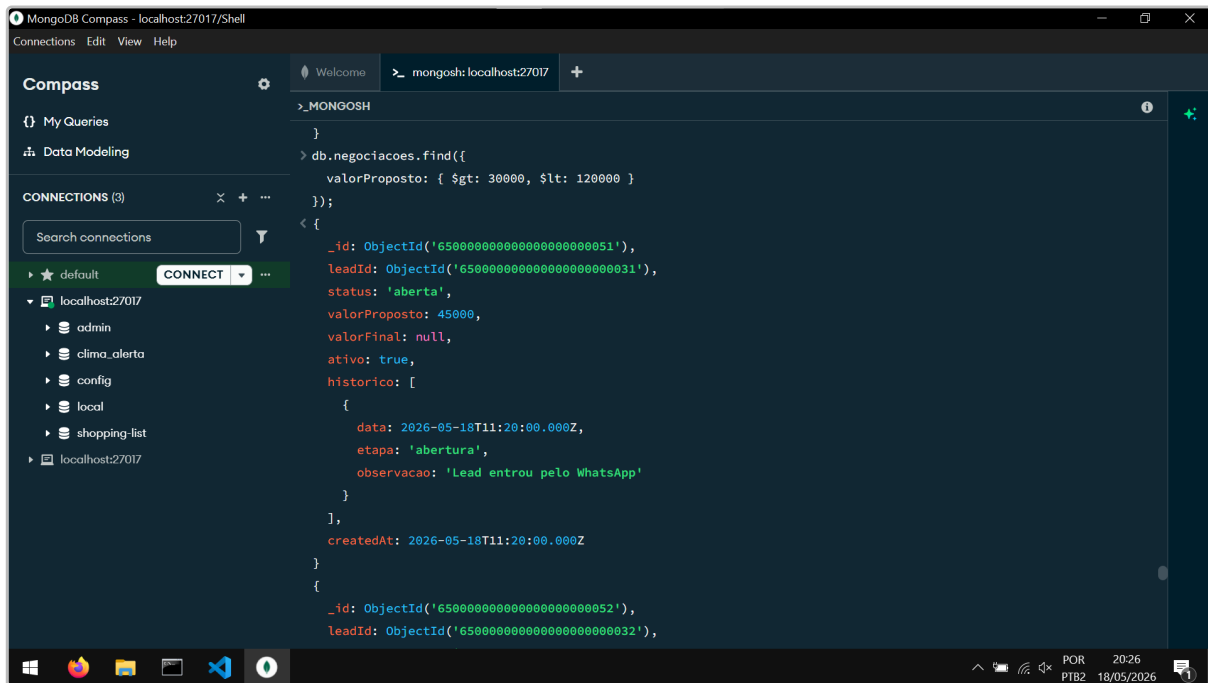


Figura 8 – Consulta com operadores `$gt` e `$lt`.

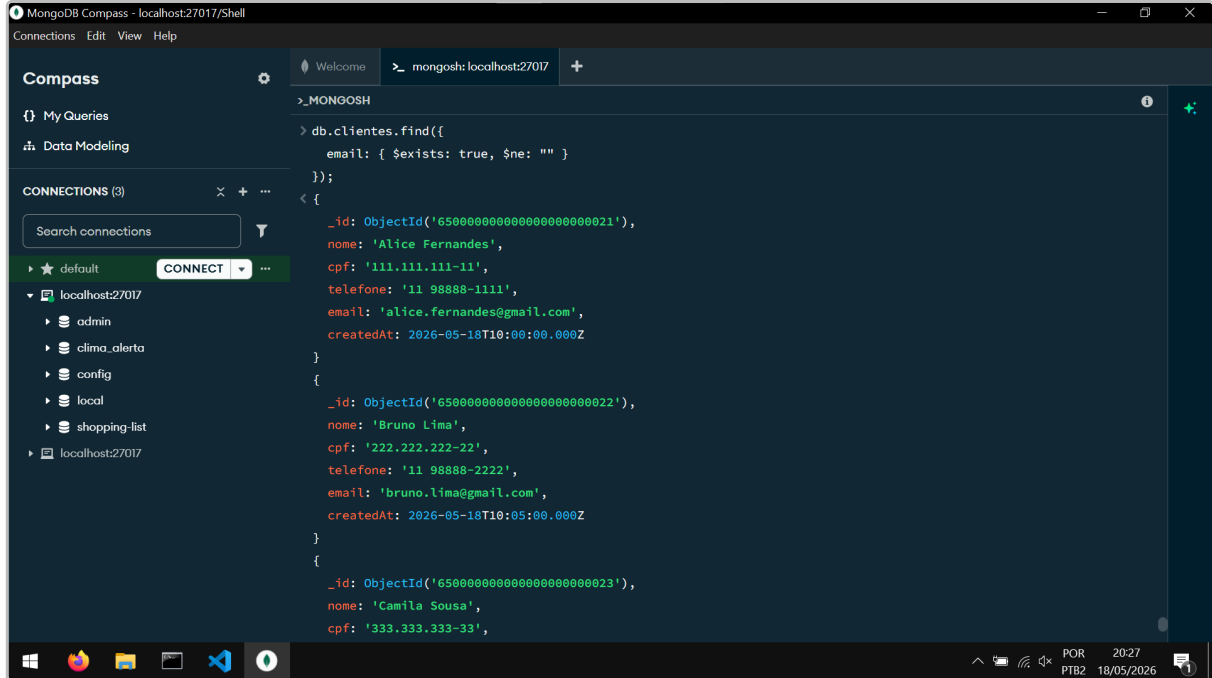
Fonte: elaborado pelos autores (2026).

- `$gt` (greater than): retorna documentos com `valorProposto` maior que 30.000;
- `$lt` (less than): retorna documentos com `valorProposto` menor que 120.000;
- a combinação dos dois operadores define um intervalo numérico fechado.

### 3.3 Operador `$exists`

A consulta valida a presença e o preenchimento do campo **email** na coleção de clientes, retornando apenas os documentos que possuem o campo preenchido com um valor não vazio.

```
db.clientes.find({
  email: { $exists: true, $ne: "" }
});
```



*Figura 9 – Consulta com operador \$exists.*

*Fonte: elaborado pelos autores (2026).*

- `$exists: true`: garante que o campo email está presente no documento;
- `$ne: ""`: exclui documentos em que o campo existe mas está vazio.



### 3.4 Projeção e Ordenação

A consulta demonstra a seleção de campos específicos por meio de projeção e a ordenação dos resultados por data de criação de forma decrescente, retornando os leads mais recentes primeiro.

```
db.leads
.find({}, { nomeLead: 1, status: 1, origem: 1, _id: 0 })
.sort({ createdAt: -1 });
```

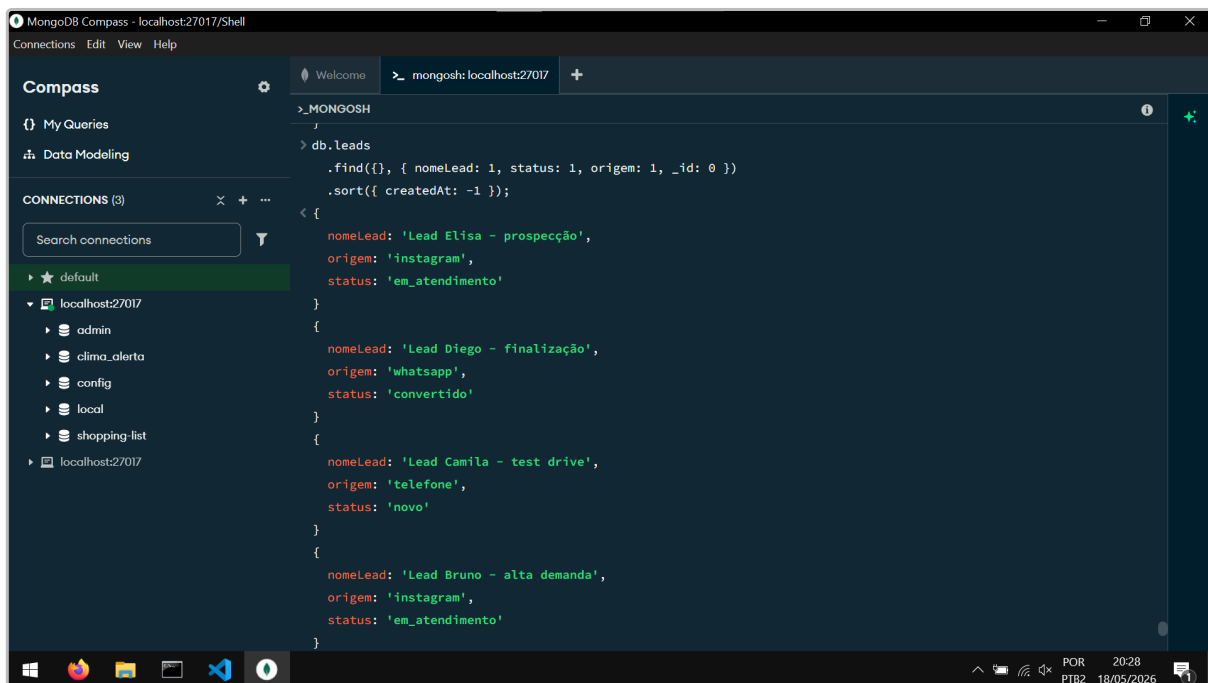


Figura 10 – Consulta com projeção e ordenação.

Fonte: elaborado pelos autores (2026).

- a projeção limita os campos retornados, reduzindo o volume de dados trafegados;
- campos com valor 1 são incluídos; `_id: 0` suprime o identificador;
- `sort({ createdAt: -1 })`: ordena do registro mais recente ao mais antigo.

### 3.5 Paginação

A consulta demonstra a técnica de paginação, essencial para interfaces que exibem grandes volumes de registros. A combinação de **skip** e **limit** define o deslocamento e o tamanho de cada página.

```
db.leads
.find({})
.sort({ createdAt: -1 })
```

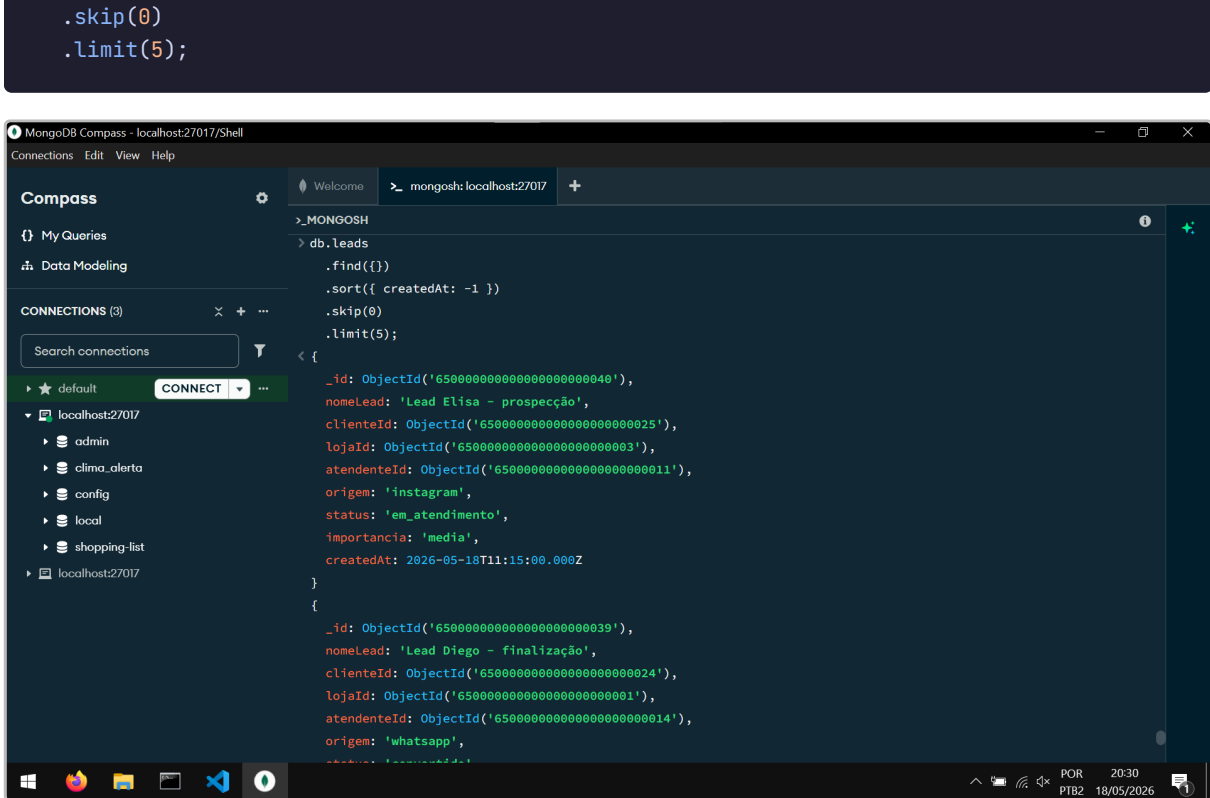


Figura 11 – Consulta com paginação.

Fonte: elaborado pelos autores (2026).

- skip(0): define o início da primeira página; para a segunda página, utilizar skip(5);
- limit(5): define o tamanho da página em cinco documentos;
- a ordenação prévia garante consistência na paginação dos resultados.

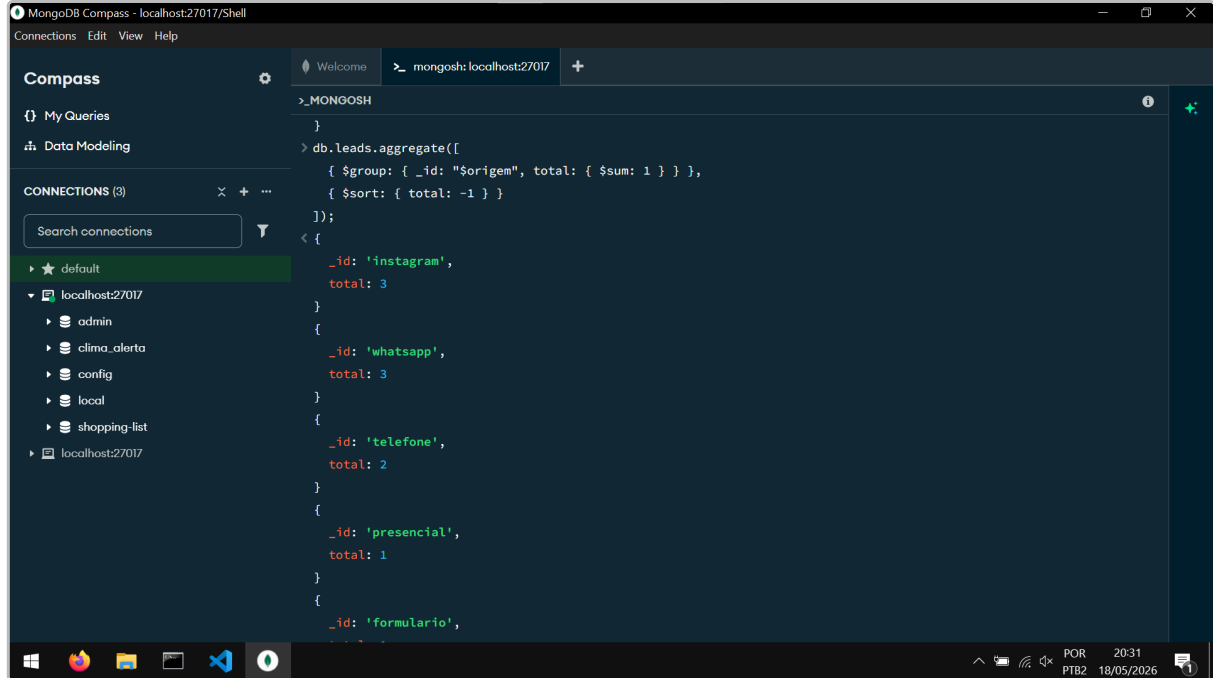
## 4 AGGREGATIONS DE DASHBOARD

As operações de aggregation geram indicadores gerenciais a partir dos leads cadastrados. O pipeline do MongoDB permite encadear estágios de transformação dos dados, produzindo métricas para o dashboard comercial da 1000 Valle Multimarcas.

### 4.1 Leads por Origem

O aggregation agrupa os leads pelo canal de captação e ordena os resultados pela quantidade de registros em ordem decrescente, identificando quais canais geram mais oportunidades.

```
db.leads.aggregate([
  { $group: { _id: "$origem", total: { $sum: 1 } } },
  { $sort: { total: -1 } }
]);
```



*Figura 12 – Aggregation de leads por origem.*

*Fonte: elaborado pelos autores (2026).*

O estágio **\$group** agrupa os documentos pelo campo **origem** e soma um ponto para cada lead. O estágio **\$sort** ordena o resultado em ordem decrescente.

## 4.2 Leads por Status

O aggregation distribui os leads pelos diferentes estágios do funil comercial, possibilitando a visualização do volume de oportunidades em cada fase do processo de vendas.

```
db.leads.aggregate([
  { $group: { _id: "$status", total: { $sum: 1 } } },
  { $sort: { total: -1 } }
]);
```

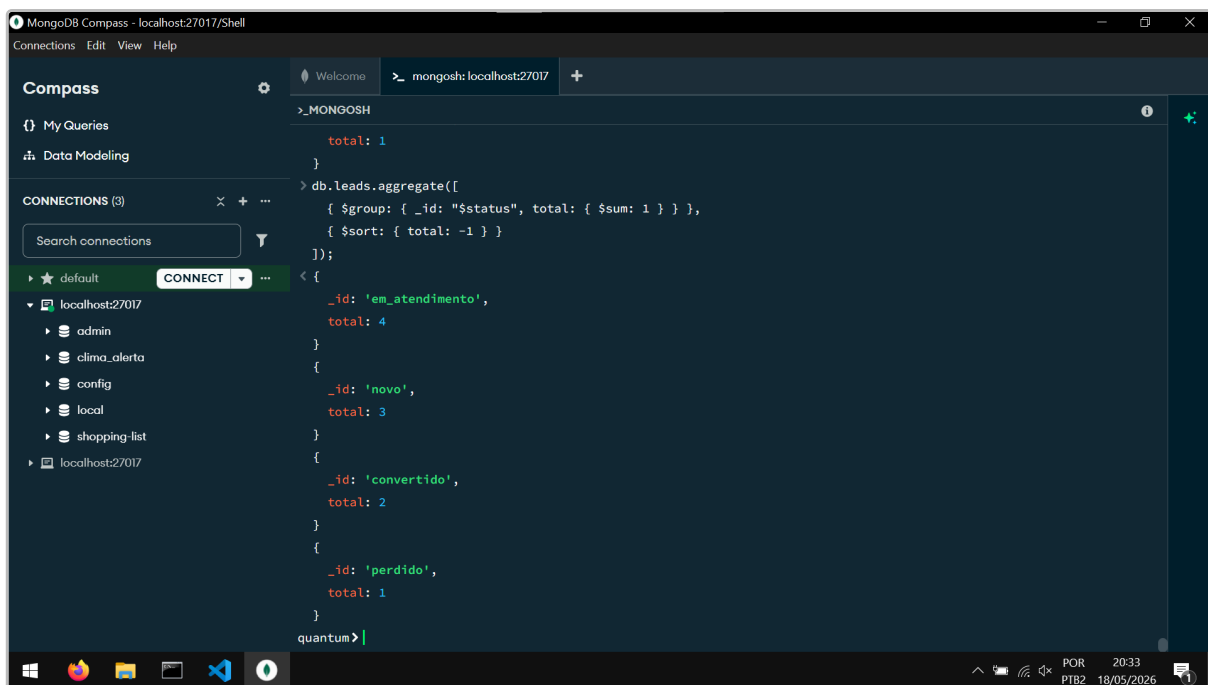


Figura 13 – Aggregation de leads por status.

Fonte: elaborado pelos autores (2026).

O resultado permite ao gerente comercial identificar gargalos no funil de vendas e redistribuir o esforço da equipe conforme necessário.

## 4.3 Taxa de Conversão

O aggregation calcula a porcentagem de leads convertidos em relação ao total de oportunidades registradas. O pipeline utiliza **\$group** para consolidar os contadores e **\$project** para calcular o percentual.

```
db.leads.aggregate([
  {
    $group: {
      _id: null,
      totalLeads: { $sum: 1 },
      convertedLeads: { $sum: { $cond: [
        { $eq: [ '$status', 'convertido' ] },
        1,
        0
      ] } }
    }
  },
  { $project: { percentual: { $divide: [ '$convertedLeads', '$totalLeads' ] } } }
]);
```

```

        totalConvertidos: {
          $sum: { $cond: [{ $eq: ["$status", "convertido"] }, 1, 0] }
        }
      },
    },
    {
      $project: {
        _id: 0,
        totalLeads: 1,
        totalConvertidos: 1,
        taxaConversaoPercentual: {
          $multiply: [{ $divide: ["$totalConvertidos", "$totalLeads"] }, 100]
        }
      }
    }
  ]
});

```

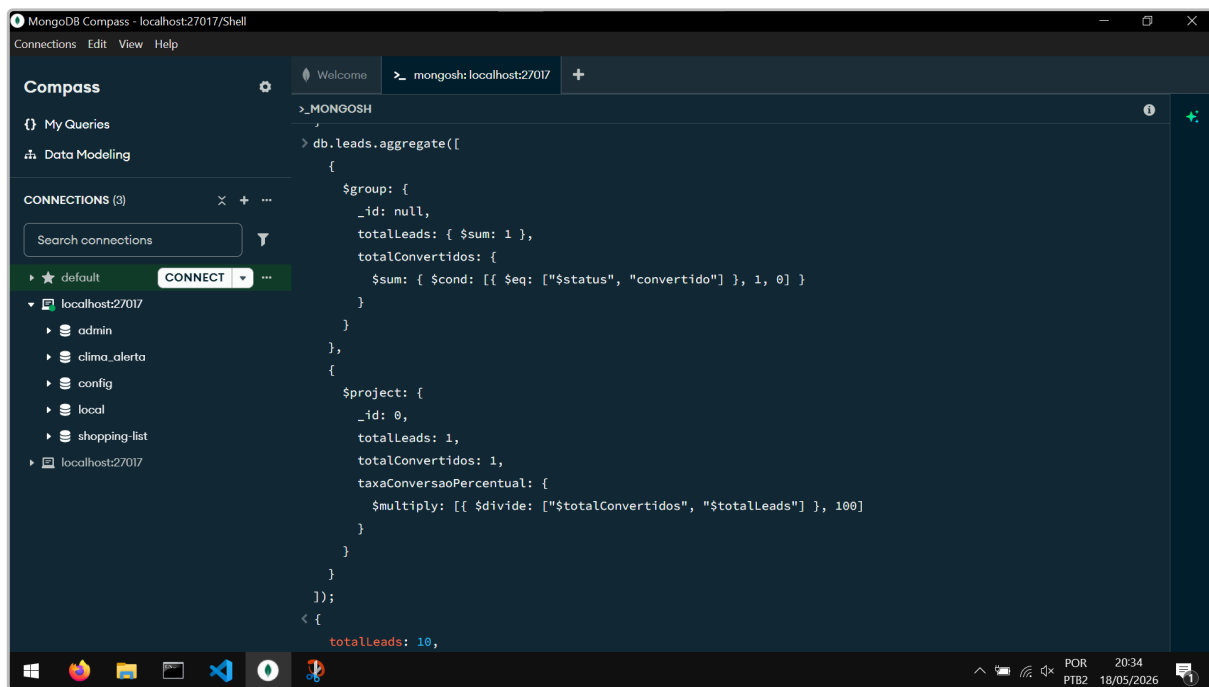


Figura 14 – Aggregation de taxa de conversão.

Fonte: elaborado pelos autores (2026).

O operador **\$cond** retorna 1 quando o status é "convertido" e 0 nos demais casos. O campo **taxaConversaoPercentual** divide os convertidos pelo total e multiplica por 100.

#### 4.4 Leads por Atendente

O aggregation contabiliza quantos leads cada atendente possui sob sua responsabilidade, permitindo ao gerente comercial avaliar a distribuição da carga de trabalho da equipe.

```

db.leads.aggregate([
  {
    $group: {
      _id: "$atendenteId",
      totalLeads: { $sum: 1 }
    }
  },

```

```
{ $sort: { totalLeads: -1 } }
});
```

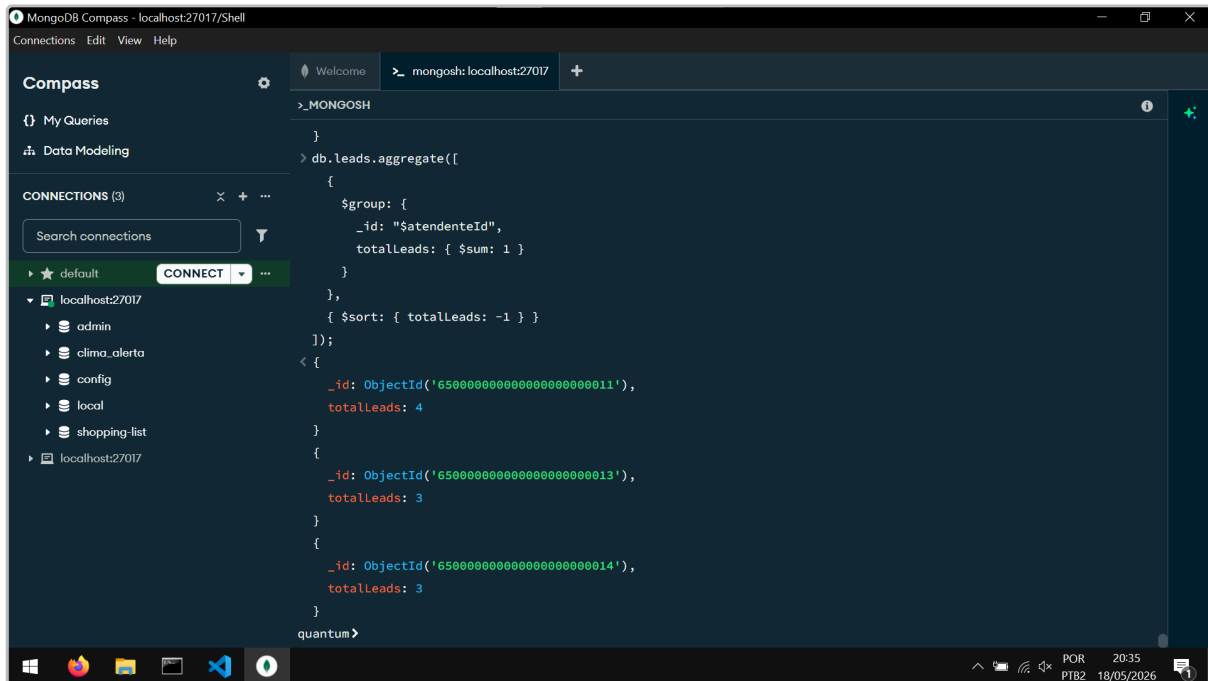


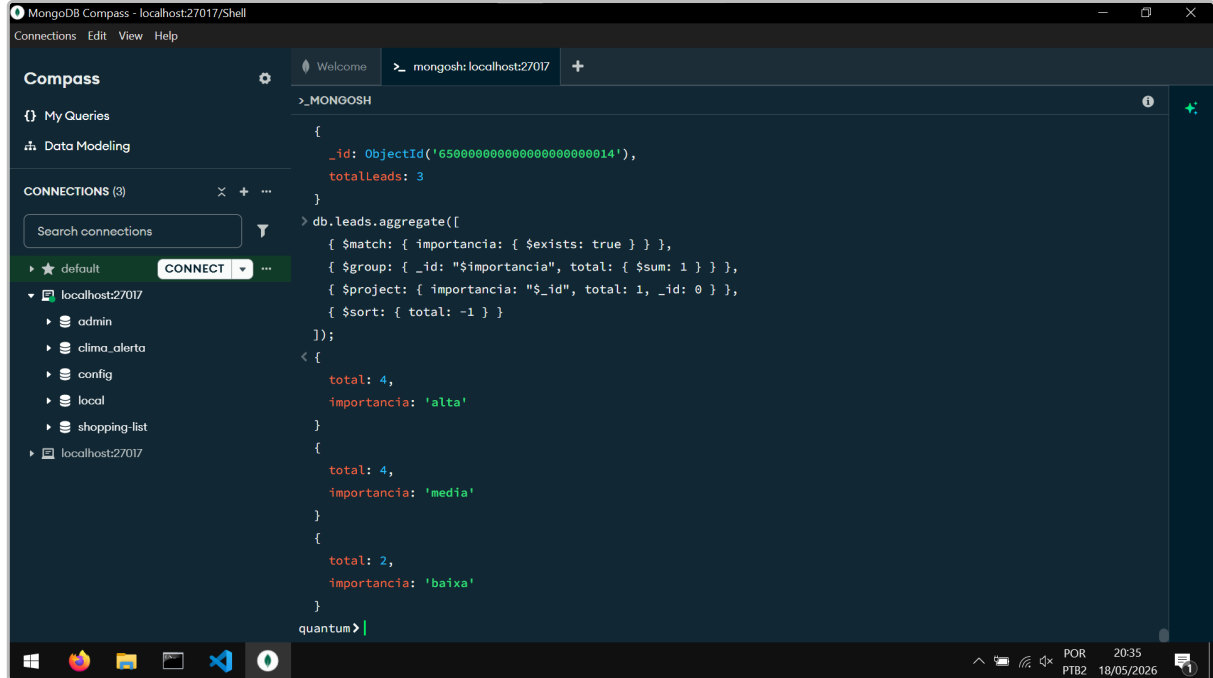
Figura 15 – Aggregation de leads por atendente.

Fonte: elaborado pelos autores (2026).

#### 4.5 Leads por Importância

O aggregation classifica os leads pelo nível de importância, auxiliando a equipe na priorização das ações comerciais. O estágio **\$match** garante que apenas documentos com o campo preenchido sejam considerados no agrupamento.

```
db.leads.aggregate([
  { $match: { importancia: { $exists: true } } },
  { $group: { _id: "$importancia", total: { $sum: 1 } } },
  { $project: { importancia: "$_id", total: 1, _id: 0 } },
  { $sort: { total: -1 } }
]);
```



*Figura 16 – Aggregation de leads por importância.*

*Fonte: elaborado pelos autores (2026).*

O estágio **\$project** renomeia o campo **\_id** para **importancia**, tornando o resultado mais legível para o dashboard gerencial.

## 5 CONCLUSÃO

O Dia 3 conclui a entrega prática da atividade de banco de dados não relacional, demonstrando a aplicação completa do modelo projetado nos dias anteriores. As atividades realizadas compreenderam:

- carga de dados mínima exigida nas seis coleções obrigatórias do projeto;
- cumprimento das regras de negócio definidas na modelagem do Dia 2, incluindo *referencing* entre entidades e *embedding* do histórico de negociação;
- execução das consultas obrigatórias com operadores lógicos (\$and, \$or), de comparação (\$gt, \$lt), de existência (\$exists), projeção, ordenação e paginação;
- geração de cinco indicadores gerenciais por meio de pipelines de aggregation, cobrindo origem, status, taxa de conversão, distribuição por atendente e priorização por importância.

O documento segue o padrão estabelecido no repositório da equipe Error Squad, com explicações detalhadas dos comandos utilizados, justificativas técnicas e referência às decisões arquiteturais documentadas no Dia 2.



